PAGE 1

Original lecture content:
SYNTAX ANALYSIS
Topic 3

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Source page contains graphical content that could not be reliably reconstructed from its text layer.]

Original lecture content:
Introduction to parsers
Syntax trees
OUTLINE
Context-free grammars
Top-down parsing
Bottom-up parsing

Original lecture content:
SYNTAX ANALYSIS
Syntax analysis recognizes the syntactic structure of the programming language and transforms a string of tokens into a tree of tokens and syntactic categories
Parser is the program that performs syntax analysis

Original lecture content:
INTRODUCTION TO PARSERS
source
Semantic
token
syntax
Scanner
Parser
Analyzer
tree
code
next token
Symbol
Table

Diagram representation / semantic relationships:
Source code -> Scanner --token--> Parser --syntax tree--> Semantic Analyzer. Parser requests next token from Scanner. Symbol Table is shared across these compiler stages.

Original lecture content:
SYNTAX ANALYZER

Diagram representation / semantic relationships:
Source expression x = a*b*c is tokenized as identifiers and operators. Syntax Analysis uses grammar S -> id = E; E -> E + T | T; T -> T * F | F; F -> id. The parse tree encodes assignment, expression and multiplication hierarchy.

Original lecture content:
Introduction to parsers
Syntax trees
OUTLINE
Context-free grammars
Top-down parsing
Bottom-up parsing

Original lecture content:
SYNTAX TREES
A syntax tree represents the syntactic structure of tokens in a program defined by the grammar of the programming language
:=
id1
+
id2
*
id3

Diagram representation / semantic relationships:
Syntax tree: := has left child id1 and right child +. The + node has left child id2 and right child *. The * node has children id3 and 60. Thus id1 := id2 + (id3 * 60).

Original lecture content:
Introduction to parsers
Syntax trees
OUTLINE
Context-free grammars
Top-down parsing
Bottom-up parsing

Original lecture content:
CONTEXT-FREE GRAMMARS (CFG)
(
V
T
P
S
)
A set of nonterminals:
syntactic categories each
of which denotes a set of
strings
A set of terminals:
basic symbols (token
types) from which
strings are formed
A set of productions: rules
specifying how the terminals
and nonterminals can be
combined to form strings
The start symbol: a
distinguished nonterminal that
denotes the whole language

Diagram representation / semantic relationships:
CFG components (V, T, P, S): V = nonterminals; T = terminals; P = production rules; S = start symbol.

Original lecture content:
PRESENTATION OF PRODUCTION RULES
N → α
Left-hand Side
- Single Non-terminal
Right-hand Side
□□
| string of terminal and/or non terminal

Diagram representation / semantic relationships:
Production N -> alpha: N is the single nonterminal on the left-hand side; alpha is the right-hand side, which may be epsilon or a string of terminals and nonterminals.

Original lecture content:
EXAMPLE 1
Given a production rules and grammar
as follows:
Identify if A □ aBb is a correct
grammar
A □ □□
B □ a
C□ aA
D □Ab

Original lecture content:
EXAMPLE 1
Identify the grammar represented by the diagram.
For each production rule, identify if it is left linear
grammar or right linear grammar.

Original lecture content:
HOMEWORK
Identify the grammar represented by the diagram.
For each production rule, identify if it is left linear
grammar or right linear grammar.

Diagram representation / semantic relationships:
Automaton: states A, B, C; initial state A; accepting state C (double circle). A --b--> A; A --a--> B; B --b--> A; B --a--> C; C --a--> C; C --b--> C. [UNVERIFIED VISUAL RELATIONSHIP: verify B-to-A curved edge label and C self-loop symbols.]

Original lecture content:
EXAMPLE 2
Given P,
S □ TaTaT
T □ aT| bT |□□
Assume V = { S, T}, T = {a,b}, S= {S }
Identify if the string babab followed the CFG.

Original lecture content:
PARSE TREES
A parse tree is a graphical representation for a derivation that filters out the order of choosing nonterminals for rewriting
Many derivations may correspond to the same parse tree, but every parse tree has associated with it a unique leftmost and a unique rightmost derivation
Parse Tree has three vertex:
A) root vertex – start symbol, Non-terminal
B) leaf vertex – Terminal
C) Intermediate vertex - NT

Original lecture content:
EXAMPLE 1
Given a grammar
S □ bSb|a|b
Draw a parse tree for string bbabb.

Original lecture content:
EXAMPLE 2
Given P
S□ AB| □□
A □ aB
B □ Sb
Derive a parse tree for string aabbb

Original lecture content:
TYPES OF DERIVATION TREES
If there are more than one nonterminal in the string, many choices are possible:
A leftmost derivation always chooses the leftmost nonterminal to rewrite the production rules
A rightmost derivation always chooses the rightmost nonterminal to rewrite the production rules

Original lecture content:
EXAMPLE 1
Given P
E □ E + E
E □ E * E
E □ a | b | c
Derive the parse tree using
the left-most derivation and
right-most derivation for
string a+b*c.

Original lecture content:
HOMEWORK
Given P
S □ T00T
T □ 0T
T □ 1T
T □ □□
Derive the parse tree using the left-most derivation and right-most derivation for string 1000111

Original lecture content:
AMBIGUOUS GRAMMARS
A grammar is ambiguous if it can derive a string with two different parse trees
If we use the syntactic structure of a parse tree to interpret the meaning of the string, the two parse trees have different meanings
Since compilers do use parse trees to derive meaning, we would prefer to have unambiguous grammars

Original lecture content:
EXAMPLE 1
S □ ABA
A □ aA |□□
B □ bB |□□
Prove that this Grammar is ambiguous using string ‘aa’

Original lecture content:
HOMEWORK
E □ E + E
E □ E * E
E □ id
Using string ‘id + id* id, prove that this grammar is ambiguous with the left right-most derivation tree

Original lecture content:
Introduction to parsers
Syntax trees
OUTLINE
Context-free grammars
Top-down parsing
Bottom-up parsing

Original lecture content:
TYPE OF PARSING
Top – down parsing
Bottom – up parsing
Parsing starts from high-level (root) to leaf node
Parsing starts from high-level (root) to leaf node
Use left most derivation
Use right most derivation
e.g Recursive Descent and LL parsers
e.g Operator Precedence Parser, LR(0) Parser, SLR Parser, LALR Parser and CLR Parser

Diagram representation / semantic relationships:
Comparison rows: top-down parsing starts at root and uses leftmost derivation; bottom-up parsing builds from leaves toward root and corresponds to reverse rightmost derivation.
[SOURCE DISCREPANCY: original bottom-up description says root to leaf, reproduced separately in original text.]

Original lecture content:
TOP-DOWN PARSING
Construct a parse tree from the root to the leaves using leftmost derivation
Example :
Consider the following grammar:
E □ TE’
E’ □ +TE’ | □□
T □ FT’
T’□ *FT’ | □□
F □ (E) | id
Create parse tree for id + id * id

Original lecture content:
TOP-DOWN PARSE
PARSING
Top-down Parse Tree for id + id * id

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Slide shows a sequence of top-down parse-tree expansion diagrams for id + id * id. The precise intermediate trees must be checked visually.]

Original lecture content:
RECURSIVE DESCENT PARSING
- a top-down parser
Rules of Recursive Descent Parsing
1.
If the input is a non-terminal, write the procedure.
2.
If the input is a terminal, compare the terminal with the input symbol. If the input symbol same with the terminal, increment the input pointer
3.
If any non-terminal has multiple productions, write all the productions in one procedure
4.
No main function and no declaration involved
- use recursive functions or procedures to parse any given string
- no involvement of backtracking (Making repeated input scans until we find a correct path.)
- the basic idea of recursive-descent parsing is to associate each non-terminal with a procedure.

Original lecture content:

```
Procedure E( )
{
T( );
Eprime ( );
}
Procedure TPrime( )
{
if (input == ‘*’)
{
input++;
F ( );
Tprime ( );
}
else
return( ); //epsilon
}
```

RECURSIVE
DESCENT PARSER

```
Procedure EPrime( )
{
if (input == ‘+’)
{
input++;
T( );
Eprime ( );
}
else
return( ): //epsilom
}
```

Consider the following grammar:

E □ TE’
E’ □ +TE’ | □□
Procedure F( )

```
{
if (input == ‘ ( ’)
{
input++;
E ( );
if (input == ‘ ( ’);
{
input++;
}
if (input == ‘ id ’)
input++;
}
}
```

T □ FT’
T’□ *FT’ | □□
F □ (E) | id
How the procedure should be
written using the Rules Decent
Parser?

```
Procedure T( )
{
F( );
Tprime ( );
}
```

Original lecture content:
HOMEWORK
Consider the following grammar:
S → if E then S else S
| begin L end
| print E
L → S ; L
| ε
E → num = num
Create the parse tree for the statement begin print num = num ; end

Original lecture content:
PREDICTIVE PARSING *UPDATED*

Original lecture content:
PREDICTIVE PARSING
Predictive parsing is a top-down parsing without backtracking
Predictive parsing is also called LL(k) parsing
The first L stands for scanning the input from left to right
The second L stands for producing a leftmost derivation
The k stands for using k lookahead input symbol to choose alternative productions at each derivation
step
Namely, according to the next token, there is only one production to choose at each derivation step
Therefore , it is also know as LL(1) parsing from now on, namely, parsing using only one lookahead input symbol (one token)

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Diagram representation / semantic relationships:
Predictive parsing six-step order: 1 Eliminate left recursion -> 2 Eliminate left factoring -> 3 FIRST and FOLLOW -> 4 Parsing table -> 5 Parse input string -> 6 Construct parse tree.

Original lecture content:
ELIMINATION OF LEFT RECURSION
Three types of recursion
Normal Recursion
Left
Right
A production of grammar is said to have normal recursion if the RHS is same as the LHS.
S →ASB
A production of grammar is said to have left recursion if the leftmost variable of its RHS is same as variable of its LHS
A production of grammar is said to have right recursion if the rightmost variable of its RHS is same as variable of its LHS
S →Sa
S → aS
Not suitable for top-down parsing, need to eliminate

Original lecture content:
WHY WE NEED TO ELIMINATE THE LEFT RECURSION
A → Aα | β
A → αA | β
Where A – NT, α β - T
From this grammar, there are two production rules:
A → Aα
From this grammar, there are two production rules:
A → αA
OR
OR
A → β
A → β
Using recursive decent parsing,
Using recursive decent parsing,
Procedure A()
We need to eliminate left
recursion because recursive
descent parsers cannot
handle rules that call
themselves before
consuming input.
it would cause infinite
recursion and prevent
successful parsing.
Procedure A()
{
{
A (); //recursive call
α;
}
α; // check first
A (); //recursive call
}

Diagram representation / semantic relationships:
Direct left recursion A -> A alpha | beta makes procedure A call itself before consuming input. Transformed right recursion A -> beta A_prime; A_prime -> alpha A_prime | epsilon consumes input before recursion.

Original lecture content:
ELIMINATION OF LEFT RECURSION – RULE 1
Left recursion is eliminated by converting the grammar into a right recursive grammar.
If we have the left-recursive pair of productions
$A \rightarrow A\alpha \mid \beta$
(Left Recursive Grammar)
where β does not begin with an A.
Then, we can eliminate left recursion by replacing the pair of productions with
$A \rightarrow \beta A'$
where A’ is a New Non Terminal
results from the elimination
$A' \rightarrow \alpha A' \mid \in$
(Right Recursive Grammar)

Original lecture content:
Rule 1
A → Aα | β
EXAMPLE 2
⇓
A → βA’
A’ → αA’ |∈
Consider the following grammar and eliminate left recursion
E → E + T | T
T → T * F |F
F → id

Original lecture content:
ELIMINATION OF LEFT RECURSION – RULE 2
If we have the left-recursive pair of productions
$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta$
Then, we can eliminate left recursion by replacing the pair of productions with
$A \rightarrow \beta A'$
$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon$

Original lecture content:
Rule 1
A → Aα | β
Rule 2
A → Aα₁ | Aα₂ | β
EXAMPLE 2
⇓
A → βA’
A’ → αA’ |∈
⇓
A → βA'
A' → α₁A' | α₂A' | ε
Consider the following grammar and eliminate left recursion
A → ABd | Aa | a
B → Be | b

Original lecture content:
ELIMINATION OF LEFT RECURSION – RULE 3
If we have the left-recursive pair of productions
A → Aα1 | Aα2 | β1 | β2
Then, we can eliminate left recursion by replacing the pair of productions with
A →β1 A' | β2 A'
A' →α1 A' | α2 A' | ε

Original lecture content:
Rule 1
A → Aα | β
Rule 2
A → $A\alpha_1$ | $A\alpha_2$ | β
Rule 3
A → Aα1 | Aα2 | β1 | β2
EXAMPLE 3
⇓
A → βA’
A’ → αA’ |∈
⇓
A → βA'
A' → α_1A' | α_2A' | ε
⇓
A →β1 A' | β2 A'
A' →α1 A' | α2 A' | ε
Consider the following grammar and eliminate left recursion
A → Ad | Ae | aB | ac
B → bBc | f

Original lecture content:
INDIRECT LEFT RECURSION
Indirect left recursion happens when a nonterminal calls itself indirectly through another nonterminal.
Consider this grammar:
A → Ba | b
B → Ac | d
there’s indirect recursion because:
For a top-down parser (like a recursive-descent parser), this still leads to infinite recursion, just like direct left recursion, because A ends up calling itself again before consuming input.
A produces Ba
B can produce Ac
if we substitute B → A c into the first rule
A →(A c) a | b
A →A c a | b

Original lecture content:
ELIMINIATION OF INDIRECT LEFT RECURSION
Example
A → B a | b
1.
Order the nonterminals: A_1, A_2,
..., An.
B → A c | d
2.
For each A_i, substitute any
production that starts with
earlier nonterminals (A_1 ...
A_{i-1}).
3.
After substitution, eliminate any
direct left recursion that
appears.
4.
Repeat for all nonterminals.

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Original lecture content:
LEFT-FACTORING
Left factoring is a process by which the grammar with common prefixes is transformed to make it useful for Top down parsers.
Example
A → ab | ac
This kind of grammar creates a problematic situation for Top down parsers.
Top down parsers can not decide which production must be chosen to parse the string since two or more rules start with the same beginning
To remove this confusion, we use left factoring.

Original lecture content:
LEFT-FACTORING

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Left-factoring example diagram or image content is not available as selectable text.]

Original lecture content:
EXAMPLE 1
Do left factoring in the following grammar-
A → aAB | aBc | aAc

Original lecture content:
EXAMPLE 2
Do left factoring in the following grammar-
S → aSSbS | aSaSb | abb | b

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Original lecture content:
FIRST AND FOLLOW
FIRST
First(α) is a set of terminal symbols that begin in strings derived from α.
Consider the production rule-
A → abc | def | ghi
Then, we have- First(A) = { a , d , g }

Original lecture content:
COMPUTING FIRST () - RULES
1.
For a production rule X → ∈, First(X) = { ∈ }
2.
For any terminal symbol ‘a’, First(a) = { a }
3.
For a production rule X →Y1Y2Y3,.. Yk, where X is a nonterminal, each Yk is a single terminal or nonterminal
Calculating First(X)
a.
If ∈∉ First(Y1), then First(X) = First(Y1)
b.
If ∈∈ First(Y1), then First(X) = { First(Y1) – ∈ } ∪ First(Y2Y3)
Calculating First(Y2Y3)
a.
If ∈∉ First(Y2), then First(Y2Y3) = First(Y2)
b.
If ∈∈ First(Y2), then First(Y2Y3) = { First(Y2) – ∈ } ∪ First(Y3)

Original lecture content:
EXAMPLE – COMPUTING FIRST
The following grammar has been eliminated for left recursion and left factor grammar.
E→TE’
E' →+TE' | □□
T →FT’
T' →*FT' | □□
F →(E) | id

Original lecture content:
FOLLOW
Follow(α) is a set of terminal symbols that appear immediately to the right of α.
COMPUTING FIRST () - RULES
1. For the start symbol S, place $ in Follow(S).
2. For any production rule A → αB, Follow(B) = Follow(A)
3. For any production rule A → αBβ,
α, β – terminal
B - nonterminal
a. If ∈∉ First(β), then Follow(B) = First(β)
b. If ∈∈ First(β), then Follow(B) = { First(β) – ∈ } ∪ Follow(A)

PAGE 54

Original lecture content:
COMPUTING FIRST () - RULES
SIMPLIFIED VERSION

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Source page contains graphical content that could not be reliably reconstructed from its text layer.]

Original lecture content:
EXAMPLE – COMPUTING FOLLOW
The following grammar has been eliminated for left recursion and left factor grammar.
E→TE’
E' →+TE' | □□
T →FT’
T' →*FT' | □□
F →(E) | id

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Original lecture content:
The first “L” in LL(1) stands for scanning the input from left to right,
The second "L“ for producing a leftmost derivation
The “1“ for using one input symbol of look ahead at each step to
make parsing action decisions.
LL (1)
PARSING
TABLE
Before LL(1) generates the parse tree, LL(1) parsing table need to
be produced first

Original lecture content:
MODEL OF LL(1) PARSER
A predictive parser has:
Stack - a sequence of grammar symbols with $ on the bottom, indicating the bottom of the stack. Initially the stack consists of the start symbol of the grammar on the top of $.
Input - the string to be parsed, followed by $, a symbol used as a right end marker to indicate the end of the input string
Parsing Table
Output

Diagram representation / semantic relationships:
LL(1) parser has an input buffer, stack, parsing table, and output actions. The top-of-stack nonterminal and current lookahead token determine the production.

Original lecture content:
STEPS FOR PREDICTIVE PARSING (STEP 4)
Algorithm to construct LL(1) Parsing Table
For each production A –> α.
1. Find First(α) and for each terminal in First(α), make entry A –> α in the table.
2. If First(α) contains ε (epsilon) as terminal, then find the Follow(A) and for each terminal in Follow(A), make entry A –> ε in the table.
3. If the First(α) contains ε and Follow(A) contains $ as terminal, then make entry A –> ε in the table for the $.

Original lecture content:
STEPS FOR PREDICTIVE PARSING (STEP 4)
First
Follow
E –> TE’
{ id, (}
{ $,) }
E’ –> +TE’/ε
{ +, ε }
{ $,) }
Step 4: Make a parsing table.
T –> FT’
{ id, (}
{ +, $,) }
T’ –> *FT’/ε
{ *, ε }
{ +, $,) }
F –> id/(E)
{ id, (}
{ *, +, $,) }
id
(
)
*
+
$
E
E’
T
T’
F

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Original lecture content:
STEPS FOR PREDICTIVE
PARSING (STEP 5)
Stack
Input
Action
Step 5: Parsing input string (using
stack push and pop operation)
Given the input string as id* id$

Diagram representation / semantic relationships:
Table-driven parsing uses stack, remaining input and action columns. Expand a nonterminal using the LL(1) table; match terminals; accept when both stack and input reach $.

Original lecture content:
Step 1: Elimination of Left Recursion
STEPS FOR
PREDICTIVE
PARSING
Step 2: Elimination of Left Factor Grammar
Step 3: Find First & Follow
Step 4: Construct Parsing Table
Step 5 : Parsing input string
Step 6 : Construct the parse tree

Original lecture content:
STEPS FOR PREDICTIVE
PARSING (STEP 6)
Step 6: Construct the parse tree

Diagram representation / semantic relationships:
Step 6 builds the parse tree from production expansions in the predictive parsing trace.

PAGE 65

Original lecture content:
-
THE END-
Don’t forget to complete Tutorial 2 and Tutorial 3

Diagram representation / semantic relationships:
[UNVERIFIED VISUAL RELATIONSHIP: Source page contains graphical content that could not be reliably reconstructed from its text layer.]